Efficient Data Structures for
RDD-based Apriori on Spark

Luis Alberto Vásquez Vargas
IC – UNICAMP

MO809 – Topics in Distributed Computing
Instructor: Prof. Luiz Fernando Bittencourt

l322989@dac.unicamp.br
April 7, 2026

Outline

1. The Base Study
2. Introduction & Motivation
3. The FIM Problem
4. Infrastructure Evolution
5. Literature Overview
6. Proposed Data Structures
7. Experimental Results
8. Conclusion
9. References

The Base Study

Original Research Paper

A Data Structure Perspective to the RDD-based Apriori Algorithm on Spark

- **Authors:** Pankaj Singh, Sudhakar Singh, P. K. Mishra, and Rakhi Garg.
- **Journal:** *International Journal of Information Technology* (Springer), 2019.
- **Focus:** Efficiency of **Hash Tree**, **Trie**, and **Hash Table Trie** in Spark.
- **Keywords:** Parallel and distributed algorithms, Big data analytics, Frequent itemset mining, Apriori, Spark, RDD.

Introduction & Motivation

- **The Goal:** Discovering hidden patterns in massive transactional datasets.
- **The Barrier: Combinatorial Explosion**
  - For a set of m items, the search space is $O(2^m)$.
  - If $m = 100$: $2^{100} \approx 1.26 \times 10^{30}$ combinations.
  - **Brute Force:** Checking every subset would take longer than the age of the universe.

Introduction & Motivation

- **Hardware Evolution:**
  - We have the "Ferrari" of computing: **Apache Spark RDDs**.
- **Software Bottleneck:**
  - Legacy structures (Hash Trees) create a **traffic jam** that wastes the speed of RAM.

The Core Thesis

Infrastructure alone cannot solve $O(2^n)$. To exploit the speed of Spark, we must replace "Disk-era" structures with memory-optimized **Tries**.

The FIM Problem

- **Frequent Itemset Mining (FIM):** Finding subsets with frequency $\geq$ *minsup*.
- **The "Bundle Deal" Analogy:**
  - If customers buy GPUs, what else do they buy?
  - $n = 1000$ items lead to 2^{1000} possible combinations.

- **The Apriori Principle:**
  "Any subset of a frequent itemset must also be frequent."
- We prune the space, but we need efficient storage for millions of candidates.

Infrastructure Evolution

Coming Soon...

Coming Soon...

Proposed Data Structures

Coming Soon...

Experimental Results

Coming Soon...

Conclusion: The Sequential Context

- **Theoretical Expectation:**
  - The **Hash Table Trie** was designed to accelerate itemset traversal by combining Trie logic with hashing.

- **Previous Practical Failure:**
  - In earlier sequential implementations, where **memory was a strict bottleneck**, the hashing overhead caused it to underperform.
  - It was often documented as the least efficient variant in non-distributed settings.

Conclusion: The Spark Paradigm Shift

- **Distributed Advantage:**
  - Within Spark's **distributed memory (RDD)** architecture, the Hash Table Trie finally realizes its theoretical efficiency.

- **Benchmark Results:**
  - Experimental data shows it performs **significantly better** than the traditional Hash Tree.
  - It maintains a slight edge over the standard Trie across most datasets.

Conclusion: Final Verdict

- **In-Memory Optimization:**
  - Memory-optimized structures are the key to high-performance FIM on Spark.

- **Summary:**
  - The **Hash Table Trie**, once considered the "worst" in sequential settings, is now the most robust choice for RDD-based Apriori.
  - Infrastructure alone (Spark) isn't enough; the underlying **data structure** must evolve to match the hardware.

References

P. Singh, S. Singh, P. K. Mishra, and R. Garg.
A data structure perspective to the RDD-based Apriori algorithm on Spark.
Int. J. Inf. Technol., 14(3):1585–1594, 2019.

Questions?

Luis Alberto Vásquez Vargas
IC – UNICAMP

l322989@dac.unicamp.br